API Security Risk Analysis

Internship program: Future Intern – Cybersecurity

Task: **Task 3**

Prepared by: **Remi Eli Kokou AKAKPO**

Role: **Cybersecurity intern**

Date: **23/03/2026**

1. Executive Summary

This report presents the results of a security analysis conducted on two public reference APIs: ReqRes (reqres.in) and JSONPlaceholder (jsonplaceholder.typicode.com). The analysis is based on official documentation published by these platforms, technical guides available at reqres.in/docs, and the OWASP API Security Top 10 (2023) framework.

ReqRes has significantly evolved in 2025-2026: the platform transitioned from a static mock API to a real REST backend with API key authentication (x-api-key), user sessions via Bearer tokens, request logs, and persistent collections. This evolution fundamentally changes the risk profile compared to previous analyses based on the old simulated behavior.

Indicator	Value	Status	Source
APIs analyzed	2 (ReqRes + JSONPlaceholder)	**Complete**	Official sources
Risks identified	7 documented vulnerabilities	**Critical**	OWASP + reqres.in docs
HIGH risks	2 / 7	**Priority**	Immediate action
MEDIUM risks	3 / 7	**Plan**	Short term
LOW risks	2 / 7	**Monitor**	Medium term

2. Context & Methodology

2.1 Overview of Tested APIs

Both APIs were selected because they are explicitly recommended by the Future Interns Task 3 (2026) assignment and represent two very different levels of security maturity, allowing a full spectrum of risks to be illustrated.

API	Base URL	Current Auth	Logs	Persistence
ReqRes	https://reqres.in/api/*	x-api-key + Bearer	Yes (Pro)	Yes (collections)
JSONPlaceholder	https://jsonplaceholder.typicode.com	None	No	No (read-only)

2.2 ReqRes Architecture (2026)

The official reqres.in documentation reveals a two-layer authentication architecture, which is a crucial security element to analyze:

Layer	Required Header	Scope	Risk if Absent
API Level	x-api-key: YOUR_KEY	/api/* endpoints - CRUD, users, collections (admin)	**Access denied (401)**
User Level	Authorization: Bearer <token>	/app/* endpoints - Per-user data (scoped)	**Access denied (401)**

2.3 Analysis Methodology

- Reading and analysis of official reqres.in documentation (docs, blog, QA guides)
- Study of exposed endpoints, accepted HTTP methods, and documented JSON response structures
- Analysis of authentication mechanisms: x-api-key, Bearer tokens, magic links, sessions
- Identification of present/absent security headers according to OWASP best practices
- Risk classification according to OWASP API Security Top 10 (2023)
- Formulation of remediation recommendations aligned with industry standards

2.4 Severity Grid

Level	Indicative CVSS	Criteria	Remediation Delay
HIGH	7.0 - 10.0	Unauthorized access, data exposure, auth bypass	**< 48 hours**
MEDIUM	4.0 - 6.9	Misconfiguration, absence of non-critical controls	**< 2 weeks**
LOW	0.1 - 3.9	Minor issues, best practices not followed	**< 1 month**

3. Analysis - JSONPlaceholder (jsonplaceholder.typicode.com)

JSONPlaceholder is a public REST API without authentication, designed for rapid prototyping and tutorials. It exposes 6 resources (posts, comments, albums, photos, todos, users) via fully open HTTP endpoints. According to its own documentation, all CRUD operations are simulated - data does not actually persist.

Exposed Endpoints and Response Structure

```
GET /users           - 10 users with complete personal data
GET /users/1         - Full detail: name, email, phone,
address, GPS, company
GET /posts           - 100 posts with userId, title, body
GET /todos           - 200 tasks with userId, title, completed
POST /users          - Simulated creation (returns 201 without
persistence)
DELETE /posts/1      - Simulated deletion (returns 200 with no
effect)
```

Sample Response — GET /users/1 (exposed data)


```
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "phone": "1-770-736-8031 x56442",
  "website": "hildegard.org",
  "address": {
    "street": "Kulas Light", "suite": "Apt. 556",
    "city": "Gwenborough", "zipcode": "92998-3874",
    "geo": { "lat": "-37.3159", "lng": "81.1496" }
  },
```

```
  "company": { "name": "Romaguera-Crona", "catchPhrase":
"Multi-layered..." }
}
```


RISK-001 - Total Absence of Authentication on All Endpoints HIGH	
OWASP Ref.	API1:2023 - Broken Object Level Authorization (BOLA)
Endpoint	GET /users, /posts, /todos, /comments
Observation	All JSONPlaceholder endpoints are accessible without any authentication mechanism. No token, no API key, no authorization header is required. Any HTTP client can read, create, modify, or delete resources without identifying itself. According to OWASP API1:2023, BOLA occurs when an API does not verify that the caller is authorized to access the requested object - here, there is simply no verification at all.
Business Impact	In a production context, this would allow any attacker to exfiltrate the entire database through simple pagination (GET /users?_limit=100). OWASP classifies BOLA as the most critical API risk because it is frequent, easy to exploit, and has high impact on data confidentiality (GDPR Articles 5 and 25 violation).
Remediation	Implement OAuth 2.0 or an API key system (as ReqRes does with x-api-key). Each endpoint must verify the caller's identity before returning data. Apply the principle of least privilege: a user should only access their own resources (GET /users/me only, not GET /users/{other_id}).

RISK-002 - Excessive Exposure of Personal Data (PII) HIGH	
OWASP Ref.	API3:2023 - Broken Object Property Level Authorization (BOPLA)
Endpoint	GET /users, GET /users/{id}
Observation	The /users response returns 12 fields per user, including: complete address with apartment number, phone number with extension, precise GPS coordinates (latitude/longitude), personal website, and company information. This sensitive data is returned identically whether it is a public or authenticated call, with no context differentiation. OWASP API3:2023 (formerly Excessive Data Exposure) covers precisely this case where the API returns more data than necessary.
Business Impact	Massive exfiltration of PII (Personally Identifiable Information) is possible without authentication. GPS coordinates enable user

	geolocation. The phone number and email facilitate targeted phishing or vishing attacks. Direct GDPR violation: potential sanctions of up to 4% of annual worldwide turnover (Article 83).
Remediation	Apply data minimization (Privacy by Design, GDPR Article 25): return only the fields necessary for the context. Create differentiated projections: a public endpoint returns only id + username, an authenticated endpoint returns details according to token rights. Remove GPS coordinates from API responses except for explicit geolocation use.

RISK-003 - Absence of Rate Limiting - Scraping and DoS Risk MEDIUM	
OWASP Ref.	API4:2023, Unrestricted Resource Consumption
Endpoint	GET /users, /posts, /comments (all endpoints)
Observation	No request limiting mechanism is present. The headers X-RateLimit-Limit, X-RateLimit-Remaining, and Retry-After are absent from responses. It is possible to send an unlimited number of requests without triggering any blocking. OWASP API4:2023 covers this case: an API that does not limit the number of requests is vulnerable to denial of service (DoS) attacks and massive automated data scraping.
Business Impact	Enables automated scraping of all data (100 posts, 10 users, 200 todos) in seconds. In a production context, this would expose the organization to mass data exfiltration or server resource saturation, rendering the service unavailable for legitimate users.
Remediation	Implement a rate limiter on all endpoints: maximum 60 requests/minute per IP for public endpoints. Add standard headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset. Return HTTP 429 (Too Many Requests) with a Retry-After header when the limit is exceeded. Use solutions such as nginx rate limiting, API Gateway, or libraries like express-rate-limit.

4. Analysis - ReqRes (reqres.in)

ReqRes has significantly evolved in 2025-2026. The platform now offers a real REST backend with genuine API key authentication, user sessions via Bearer tokens, persistent collections, and request logs. This evolution represents a significant improvement in security posture compared to previous versions.

Documented Authentication Flow (source: reqres.in/blog/api-testing-with-postman)

```
# Step 1 — Request with x-api-key (endpoints /api/*)
GET https://reqres.in/api/users?page=2
    -H "x-api-key: YOUR_API_KEY"
    -H "Content-Type: application/json"

# Step 2 — Create a magic link (user auth)
POST https://reqres.in/api/auth/magic-link
    -H "x-api-key: YOUR_API_KEY"
    -d '{"email": "user@example.com"}'

# Step 3 — Verify and obtain session token
POST https://reqres.in/api/auth/verify
    -d '{"token": "MAGIC_LINK_TOKEN"}'

# Step 4 — Access user data (/app/*)
GET https://reqres.in/app/collections/todos/records
    -H "Authorization: Bearer YOUR_SESSION_TOKEN"
```

Response Structure — GET /api/users (source: reqres.in/docs)

```
{
  "page": 2,
  "per_page": 6,
  "total": 12,
  "total_pages": 2,
  "data": [
    { "id": 7, "email": "michael.lawson@reqres.in",
      "first_name": "Michael", "last_name": "Lawson",
      "avatar": "https://reqres.in/img/faces/7-image.jpg" }
  ]
}
```

RISK-004 - Auth Header Confusion - Risk of Unauthorized Access MEDIUM	
OWASP Ref.	API2:2023 - Broken Authentication
Endpoint	GET /app/* with x-api-key instead of Bearer
Observation	The official documentation (reqres.in/blog/qa-automation-with-reqres) states that confusion between the two authentication modes is the most frequent cause of 401 and 403 errors. The /app/* endpoints mandatorily require a user session Bearer token, sending an x-api-key returns 401 Unauthorized. This separation, while architecturally correct, creates a risk of improper client-side implementation: a misconfigured application might send requests without the correct headers, exposing data or creating access gaps.
Business Impact	A misconfigured client application could attempt to access /app/* endpoints with the wrong token type, either receiving 401 errors that reveal the API structure, or, if validation is not strict, accessing another user's data. The documentation itself acknowledges that this confusion is the most common source of errors.
Remediation	Clearly document the header/endpoint mapping throughout all internal documentation. Implement precise error messages (without revealing internal structure): 'This endpoint requires a user session token'. Add automated tests verifying that each header type is rejected on the wrong endpoints. Use an API Gateway to centralize authentication management.

RISK-005 - Rate Limiting - Handling 429s in CI/CD MEDIUM	
OWASP Ref.	API4:2023 - Unrestricted Resource Consumption
Endpoint	All endpoints - automated CI runners
Observation	The official documentation (reqres.in/blog/qa-automation-with-reqres) explicitly mentions that CI runners can trigger rate limits or WAF (Web Application Firewall) rules via Cloudflare. HTTP 429 is returned when the limit is exceeded. Retry-After headers are present but must be actively respected by clients. This risk is bidirectional: an API without rate limiting is exposed to abuse, but a client that does not respect rate limiting can cause service interruptions.
Business Impact	CI/CD pipelines sending burst requests can be blocked by the Cloudflare WAF (403 Forbidden), interrupting deployments. In a production environment, failure to respect rate limiting can result in IP bans, affecting all users in the organization sharing the same IP address.

Remediation	Implement exponential backoff on all API requests in CI: 200-500ms delay between requests, doubling the delay with each 429/503 error. Respect the Retry-After header when present. Use a stable and identifiable User-Agent (e.g., reqres-qa-tests/1.0) to facilitate debugging. Limit API test parallelism in CI to a maximum of 5 simultaneous requests.

RISK-006 - Free Plan Limit - Risk of Service Disruption LOW	
OWASP Ref.	API8:2023 - Security Misconfiguration
Endpoint	All endpoints - Free plan (250 req/day)
Observation	The ReqRes free plan is limited to 250 requests per day and 1 API key. This limit, officially documented (reqres.in/for/qa-engineers), represents an availability risk in an intensive testing environment. A poorly calibrated automated test suite can exhaust the daily quota in a few minutes, rendering the service unavailable for the rest of the day.
Business Impact	In a team development context, quota exhaustion blocks all developers using the same API key. CI/CD pipelines stop. Regression tests can no longer run. This availability risk, although not related to a security vulnerability, directly impacts the productivity and reliability of delivery processes.
Remediation	Use separate API keys per environment (dev, staging, CI) and per developer. Monitor daily API consumption. Implement a local cache for identical repetitive requests. Consider upgrading to a paid plan for teams with intensive needs. Document the limits in the project README to raise team awareness.

5. ReqRes vs JSONPlaceholder - Security Maturity Comparison

Source: reqres.in/for/qa-engineers - Official ReqRes vs JSONPlaceholder comparison table

Security Feature	ReqRes (2026)	JSONPlaceholder	Risk if Absent
Authentication	x-api-key + Bearer	None	Unauthorized access
Real Auth Flow	Magic link session token	Non-existent	Impossible to test
Persistent Data	JSON Collections	Simulated only	False security
Request Logs	Included (Pro)	None	No traceability
User Isolation	Per-user scoping	Shared data	BOLA possible
Rate Limiting	WAF Cloudflare + 429	Not documented	Scraping/DoS
PII Exposure	Minimal (5 fields)	Excessive (12+ fields, GPS)	GDPR Violation
Overall Security Score	**MEDIUM-GOOD**	**INSUFFICIENT**	-

5.1 7-Risk Summary Table

ID	Title	API	OWASP 2023	Severity	Priority
001	Absence of authentication	JSONPlaceholder	API1 - BOLA	**HIGH**	Immediate
002	Excessive PII exposure	JSONPlaceholder	API3 - BOPLA	**HIGH**	Immediate
003	No rate limiting	JSONPlaceholder	API4 - Resource	**MEDIUM**	< 2 wks
004	Auth header confusion	ReqRes	API2 - Broken Auth	**MEDIUM**	< 2 wks
005	CI/CD rate limit	ReqRes	API4 - Resource	**MEDIUM**	< 2 wks

ID	Title	API	OWASP 2023	Severity	Priority
006	Free plan quota	ReqRes	API8 - Misconfig	**LOW**	< 1 mo.
007	No logs (free plan)	ReqRes	API8 - Misconfig	**LOW**	< 1 mo.

6. Remediation Plan & Recommendations

6.1 Priority Actions (< 48h) - HIGH Risks

- RISK-001: Implement authentication on all endpoints. ReqRes demonstrates the right approach: x-api-key for admin endpoints (/api/*) and Bearer token for user data (/app/*). No resource should be accessible without valid authentication.
- RISK-002: Apply data minimization. ReqRes returns only 5 fields (id, email, first_name, last_name, avatar) vs 12+ for JSONPlaceholder. Adopt this model: return only the fields strictly necessary for the call context.

6.2 Short-Term Actions (< 2 weeks) - MEDIUM Risks

- RISK-003: Deploy a rate limiter on all public endpoints. Add headers X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset. Return HTTP 429 with Retry-After when the limit is exceeded.
- RISK-004: Clearly document the header/endpoint mapping. Automatically test that each header type is rejected on the wrong endpoints (x-api-key rejected on /app/*, Bearer rejected on /api/* without key).
- RISK-005: Implement exponential backoff in all automated API clients. Respect the Retry-After header. Limit CI parallelism to 5 simultaneous requests.

6.3 Medium-Term Actions (< 1 month) - LOW Risks

- RISK-006: Use separate API keys per environment and developer. Monitor consumption. Document limits in the README.
- RISK-007: Enable request logs (Pro plan) in staging and production environments to allow debugging and anomaly detection.

6.4 Recommended Target Configuration - Security Headers

Every production API endpoint must return these headers:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Strict-Transport-Security: max-age=31536000;
includeSubDomains; preload
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Cache-Control: no-store, no-cache, must-revalidate
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 59
X-RateLimit-Reset: 1742774400

# MUST BE REMOVED:
# X-Powered-By(reveals backend framework)
# Server: Express(reveals the server)
```

6.5 API Security Checklist - Do's & Don'ts

#	DO's	DON'Ts
1	Use x-api-key for admin endpoints and Bearer for user endpoints (ReqRes model)	Never expose an endpoint without authentication in production
2	Return only necessary fields (data minimization, ReqRes model: 5 fields)	Never return GPS coordinates, phone number, and full address in a public response
3	Implement rate limiting with X-RateLimit-* headers on all endpoints	Never leave an endpoint consuming resources without a limit
4	Enable request logs for traceability and debugging (available in ReqRes Pro)	Never deploy an API to production without request traceability
5	Isolate data per user with strict per-user data isolation	Never allow a user to access another user's data

7. Conclusion

This comparative analysis of ReqRes and JSONPlaceholder, based on the official sources of these platforms and the OWASP API Security Top 10 (2023) framework, highlights a striking contrast between two radically different approaches to API security.

JSONPlaceholder, despite its undeniable educational utility, presents the three most critical vulnerabilities that a production API should never have: total absence of authentication (API1:2023 BOLA), excessive exposure of personal data including GPS coordinates (API3:2023), and absence of rate limiting (API4:2023). These three combined risks would constitute, in a real environment, serious GDPR violations and immediately exploitable attack vectors.

ReqRes, in its 2026 version, demonstrates significantly superior security maturity: dual-layer authentication (x-api-key + Bearer), minimized data exposure (5 fields vs 12+), per-user isolation, rate limiting via Cloudflare WAF, and request logs. The risks identified on ReqRes are operational in nature (header confusion, quota management) rather than architectural, which demonstrates a solid security design.

- Lesson 1: JSONPlaceholder is ideal for learning but represents a complete security anti-pattern. Any production API should do the exact opposite of JSONPlaceholder on the three axes of authentication / minimization / rate limiting.
- Lesson 2: ReqRes illustrates that security and ease of use are not incompatible, an API can be simple for developers to use while implementing robust authentication mechanisms.
- Lesson 3: The OWASP API Security Top 10 (2023) remains the de facto framework for classifying and prioritizing API risks. The 7 risks identified in this report cover API1, API2, API3, API4, and API8 of the Top 10.

Final Recommendation

API security rests on four inseparable pillars: Strong Authentication (API key + session tokens), Granular Authorization (per-user isolation), Resource Protection (rate limiting, WAF), and Exposure Minimization (data, errors, technical headers). ReqRes 2026 implements all four pillars, JSONPlaceholder implements none. Any API exposed in production smust align with the ReqRes model, not the JSONPlaceholder model.

Sources Used	Mission Framework
reqres.in/docs, reqres.in/blog, reqres.in/for/qa-engineers OWASP API Security Top 10 (2023), owasp.org	Future Interns, Cybersecurity Task 3 (2026) Ref. FI-CYBER-2026-003 | March 23, 2026 | CONFIDENTIAL